

Codice formazione API-fetch

Di seguito verranno riportati tutti gli script utilizzati per la creazione della formazione backend JEMORE su API e fetch:

- **File** `schema.sql` , contenente le caratteristiche del database utilizzato:

```
CREATE TABLE IF NOT EXISTS posts(  
  id SERIAL PRIMARY KEY,  
  title VARCHAR(255) NOT NULL,  
  content TEXT NOT NULL  
);  
  
INSERT INTO posts (title, content) VALUES  
( 'Post 1', 'Content of post 1'),  
( 'Post 2', 'Content of post 2'),  
( 'Post 3', 'Content of post 3');
```

- **Pool di connessione**, utile per ottimizzare le modalità di connessione del nostro sito con il database

```
```lib\db.ts```  
import { Pool } from "pg";

declare global {
 var _postgresPool: Pool | undefined
}

class PostgresDB {
 private static instance: PostgresDB
 public pool: Pool

 private constructor() {
 if (!process.env.DATABASE_URL) {
```

```

 throw new Error('POSTGRES_URL environment variable is not defined')
 }

 // Configurazione con connection string
 this.pool = global._postgresPool || new Pool({
 connectionString: process.env.DATABASE_URL,
 max: 20, // numero massimo di client nel pool
 idleTimeoutMillis: 30000,
 connectionTimeoutMillis: 2000,
 ssl: process.env.NODE_ENV === 'production' ?
 { rejectUnauthorized: false } : false
 })

 // In sviluppo, mantieni il pool in globalThis
 if (process.env.NODE_ENV === 'development' && !global._postgresPool) {
 global._postgresPool = this.pool
 }

 this.pool.on('error', (err) => {
 console.error('Unexpected error on idle PostgreSQL client', err)
 })
}

public static getInstance(): PostgresDB {
 if (!PostgresDB.instance) {
 PostgresDB.instance = new PostgresDB()
 }
 return PostgresDB.instance
}

public async query(sql: string, params?: any[]) {
 const client = await this.pool.connect()
 try {
 return await client.query(sql, params)
 } finally {

```

```

 client.release()
 }
}

public async close() {
 await this.pool.end()
 global._postgresPool = undefined
}
}

const db = PostgresDB.getInstance()

// Chiudi pulitamente alla terminazione dell'app
process.on('beforeExit', async () => {
 await db.close()
})

export { db }

```

- **Route GET** per trovare tutti gli articoli presenti, viene utilizzata nel momento in cui il nostro sito si apre per caricare tutti gli articoli salvati sul database.

```

``/api/blog/route.ts``
import { NextResponse } from "next/server";
import { db } from "../../lib/db";

// Funzione per ottenere tutti i post
export async function GET() {
 try {
 const result = await db.query("SELECT * FROM posts");
 return NextResponse.json(result.rows, { status: 200 });
 } catch (error) {
 console.error("Error fetching posts:", error);
 return NextResponse.json({ error: "Failed to fetch posts" },

```

```
{ status: 500 }));
}
}
```

- **Fetch della route** GET

```
``/utils/posts``
// GET ALL
export async function fetchPosts() {
 try {
 const res = await fetch("/api/blog") // get a blog

 if (!res.ok) { //se va male...
 console.error("Errore durante la fetch");
 }

 const data = await res.json(); //salvo i dati sottoforma di json
 return data;
 } catch (error) {
 console.error("Errore nella fetch:", error);
 return [];
 }
}
```

- Utilizzo della funzione di fetch della route GET nel frontend per caricare gli articoli.

```
``/components/blogList.tsx``
import { fetchPosts } from "../utils/posts";

export default function BlogList() {
 const [showPopup, setShowPopup] = useState(false);
 const [posts, setPosts] = useState([]);
 //posts come uno stato → array vuoto
```

```

useEffect(() => { // arrow o lambda function
 // le pagine client side non possono
 // essere asincrone, per cui creiamo una funzione
 // asincrona che prenda i dati
 async function loadPosts() {
 const data = await fetchPosts();
 setPosts(data);
 }

 loadPosts();

}, [])//array delle dipendenze vuoto

```

- **Route** **POST** per creare un nuovo articolo con titolo e contenuto forniti.

```

``/api/blog/route.ts``
import { NextResponse } from "next/server";
import { db } from "../../lib/db";
// Funzione per aggiungere un post
export async function POST(request: Request) {
 try {
 const body = await request.json();

 // Validazione dei dati
 if (!body.title || !body.content) {
 return NextResponse.json(
 { error: "Title and content are required" },
 { status: 400 }
);
 }

 // Aggiungi il nuovo post
 const result = await db.query(
 "INSERT INTO posts (title, content) VALUES ($1, $2) RETURNING *"
);
 }
}

```

```

 [body.title, body.content]
);

 // Salva il post in un file o in un database (simulato qui con un array)
 return NextResponse.json({ message: "Post added successfully", post:
result.rows[0] }, { status: 201 });
} catch (error) {
 return NextResponse.json({ error: "Failed to add post" }, { status: 500
});
}
}

```

- Funzione di fetch della route `POST` , viene chiamata nel momento in cui premiamo il bottone di conferma.

```

``/utils/posts``
//ADD
export async function addPost({ title, content }: { title: string; content: string }) {
 try {
 const res = await fetch("/api/blog", {
 method: "POST",
 headers: {
 "Content-Type": "application/json",
 },
 body: JSON.stringify({ title, content }),
 });

 if (!res.ok) {
 const errorData = await res.json();
 throw new Error(errorData.error || "Errore durante l'aggiunta del post");
 }

 const data = await res.json();
 }
}

```

```

 return data; // contiene { message, post }
 } catch (error) {
 console.error("Errore nella POST:", error);
 return null;
 }
}

```

- Utilizzo della funzione di fetch, in una funzione che viene chiamata nel momento viene inviata la conferma dell'operazione di aggiunta di un nuovo articolo.

```

``app/components/popup.tsx``
import { useState } from "react";
import { addPost, updatePost } from "../utils/posts";

export default function Popup({ onClose, postId }: Props) {
 const [title, setTitle] = useState<string>("");
 const [content, setContent] = useState<string>("");

 const handleSubmit = async (e: React.FormEvent) => {
 e.preventDefault();
 // Qui puoi gestire l'invio dei dati al server o qualsiasi altra logica necessaria
 const result = postId ? await updatePost(postId, title, content) :
 await addPost({ title, content });
 console.log({ result })
 if (result) {
 alert("Operazione andata a buon fine");
 setTitle("");
 setContent("");
 window.location.href = "/blog";
 } else {
 alert("Errore durante l'aggiunta del post.");
 }
 }
}

```

```
onClose();
};
```

- **Route** **GET** per prendere il singolo articolo con id fornito, necessaria nel momento in cui andiamo a selezionare un articolo nel sito.

```
``/api/blog/[id]/route.ts``
// Funzione per ottenere un post specifico
export async function GET(request: Request, context: { params: { id: string } }) {
 try {
 const params = await context.params;
 const id = Number(params.id);

 // Validazione dell'ID
 if (!id) {
 return NextResponse.json(
 { error: "ID is required" },
 { status: 400 }
);
 }

 // Ottieni il post
 const result = await db.query("SELECT * FROM posts WHERE id = $1",
[id]);

 if (result.rowCount === 0) {
 return NextResponse.json({ error: "Post not found" }, { status: 404 });
 }

 return NextResponse.json(result.rows[0], { status: 200 });
 } catch (error) {
 return NextResponse.json({ error: "Failed to fetch post" }, { status: 500 });
 }
}
```



- Funzione di fetch della route `GET` per selezionare il singolo articolo.

```
``/utils/posts``
// GET ONE
export async function fetchPostById(id: string) {
 try {
 const res = await fetch(`/api/blog/${id}`);

 if (!res.ok) {
 throw console.log(`Errore durante il recupero del post con ID ${id}
`);
 }

 const post = await res.json();
 return post;
 } catch (error) {
 console.log("Errore nella fetch del post:", error);
 return null; // fallback (es: se vuoi gestire l'errore nel componente)
 }
}
```

- Utilizzo della funzione di fetch, che viene chiamata nel momento in cui clicchiamo su un articolo.

```
``/app/blog/[id]/page.tsx``
import { use, useEffect, useState } from "react";
import Link from "next/link";
import { deletePostById, fetchPostById, fetchPosts } from "@app/utils/po
sts";

const PostPage = ({ params }: { params: Promise<{ id: string }> }) => {
 const { id } = use(params); // ← unwrap della Promise con React.use()
 const [showPopup, setShowPopup] = useState(false);
 const [post, setPost] = useState<any>(null);
```

```
useEffect(() => {
 async function loadPost() {
 const data = await fetchPostById(id);
 setPost(data);
 }

 loadPost();
}, [id]); // al variare di ID cambia l'articolo
```

- **Route PUT** per modificare un articolo con id fornito, come per la **POST** vista sopra viene utilizzata quando l'utente fornisce una conferma all'operazione di modifica di un articolo.

```
``/api/blog/[id]/route.ts``
// Funzione per modificare un post
export async function PUT(request: Request, context: { params: { id: string } }) {
 try {
 const params = await context.params;
 const id = Number(params.id);
 const body = await request.json();

 // Validazione dei dati
 if (!id || !body.title || !body.content) {
 return NextResponse.json(
 { error: "ID, title, and content are required" },
 { status: 400 }
);
 }

 // Aggiorna il post
 const result = await db.query(
 "UPDATE posts SET title = $1, content = $2 WHERE id = $3 RETURNING *",
 [body.title, body.content, id]
);
 }
}
```

```

);
if (result.rowCount === 0) {
 return NextResponse.json({ error: "Post not found" }, { status: 400 });
}

return NextResponse.json(
 { message: "Post updated successfully", post: result.rows[0] },
 { status: 200 }
);
} catch (error) {
 return NextResponse.json({ error: "Failed to update post" }, { status: 500 });
}
}

```

- Funzione di fetch della route **PUT** di modifica di un account

```

``/utils/posts``
//MODIFY
export async function updatePost(id: string, title: string, content: string) {
 try {
 const res = await fetch(`/api/blog/${id}`, {
 method: "PUT",
 headers: {
 "Content-Type": "application/json",
 },
 body: JSON.stringify({ title, content }),
 });

 if (!res.ok) {
 const errorData = await res.json();
 throw new Error(errorData.error || "Errore durante l'aggiornamento del post");
 }
 }
}

```

```

 const data = await res.json();
 return data; // { message, post }
 } catch (error) {
 console.error("Errore nella PUT:", error);
 return null;
 }
}

```

- Utilizzo della funzione di fetch, nel momento in cui viene cliccato il tasto Conferma nel popup

```

``app/components/popup.tsx``
import { useState } from "react";
import { addPost, updatePost } from "../utils/posts";

type Props = {
 onClose: () => void;
 postId?: string
};

export default function Popup({ onClose, postId }: Props) {
 const [title, setTitle] = useState<string>("");
 const [content, setContent] = useState<string>("");

 const handleSubmit = async (e: React.FormEvent) => {
 e.preventDefault();
 // Qui puoi gestire l'invio dei dati al server o qualsiasi altra logica necessaria
 const result = postId ? await updatePost(postId, title, content) : await addPost({ title, content });
 console.log({ result });
 if (result) {
 alert("Operazione andata a buon fine");
 setTitle("");
 setContent("");
 }
 }
}

```

```

 window.location.href = "/blog";
 } else {
 alert("Errore durante l'aggiunta del post.");
 }
 onClose();
};

```

- **Route DELETE** per eliminare un articolo con id dato

```

``/api/blog/[id]/route.ts``
// Funzione per eliminare un post
export async function DELETE(request: Request, context: { params: { id: string } }) {
 try {
 const params = await context.params;
 const id = Number(params.id);

 // Validazione dell'ID
 if (!id) {
 return NextResponse.json(
 { error: "ID is required" },
 { status: 400 }
);
 }

 // Elimina il post
 const result = await db.query("DELETE FROM posts WHERE id = $1 RETURNING *", [id]);

 if (result.rowCount === 0) {
 return NextResponse.json({ error: "Post not found" }, { status: 404 });
 }

 return NextResponse.json(
 { message: "Post deleted successfully", post: result.rows[0] },

```

```

 { status: 200 }
);
} catch (error) {
 return NextResponse.json({ error: "Failed to delete post" }, { status: 500
});
}
}

```

- Funzione di fetch della route di **DELETE** .

```

``/utils/posts``
// DELETTE ONE
export async function deletePostById(id: string) {
 try {
 const res = await fetch(`/api/blog/${id}`, {
 method: "DELETE",
 });

 if (!res.ok) {
 throw new Error(`Errore durante l'eliminazione del post con ID ${id}`);
 }

 const result = await res.json();
 return result; // contiene messaggio e il post eliminato
 } catch (error) {
 console.error("Errore nella DELETE:", error);
 return null;
 }
}

```

- Applicazione della funzione di fetch, nel momento in cui viene selezionato il pulsante cancella nel popup

```

``/app/blog/[id]/page.tsx``
import { use, useEffect, useState } from "react";

```

```
import Link from "next/link";
import { deletePostById, fetchPostById, fetchPosts } from "@app/utils/posts";
import Popup from "@app/components/popup";

const handleDelete = async () => {
 const conferma = confirm("Sei sicuro di voler eliminare questo post?");
 if (conferma) {

 const result = await deletePostById(id);
 if (result) {
 alert("Post eliminato con successo!");
 // puoi ricaricare la lista o redirect
 } else {
 alert("Errore durante l'eliminazione del post.");
 }

 window.location.href = "/blog"; // redireziona
 }
};
```